Laboratory Manual Activity No. 9

Continuing the Flutter Web Project with a Report Form and UI State Responses

Reusable Screen and Widget Structure | Loading, Success, Empty, and Error States

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

Course: IT 110 - Web Systems and Technologies

Instructor: JEAN VILLASIN, MIT

Student / Group Member: andymarkcrasco

Project: activity8 - Flutter Web (EventVenue Caraga)

Checkpoint date: October 2026

1. Activity 9 Summary

Activity 9 continues the same functioning Flutter Web project and GitHub repository created in Activity 8 - no new project was initialized. The Activity 8 Home, Login and Dashboard screens and the reusable widgets (app_button.dart, app_text_field.dart) were kept intact. On top of that baseline the group added lib/screens/report_form_screen.dart, a working Report Form for the EventVenue Caraga system where a client or vendor can report a venue listing issue, a booking problem, a payment concern or a suggestion. The form reuses the shared AppTextField and AppButton components (also used in login_screen.dart, home_screen.dart and dashboard_screen.dart) and adds field-level validation, a simulated asynchronous submit, and the four essential Flutter Web UI state responses: LOADING, SUCCESS, EMPTY and ERROR. All asynchronous responses are SIMULATED with Future.delayed because the REST API and PostgreSQL integration is planned for a later phase; the simulated failure is triggered from a clearly labeled 'Simulate API failure (demo only)' switch so the ERROR state and its Retry action can be reproduced live during the demonstration.

2. What Was Added to the Activity 8 Project

- lib/screens/report_form_screen.dart (NEW) - form, validation, and the four async UI states.
- lib/widgets/app_state_view.dart (NEW) - reusable renderer for LOADING / SUCCESS / EMPTY / ERROR.
- lib/services/report_service.dart (NEW) - simulated async submit, can simulate an HTTP 500 failure.
- lib/models/report_model.dart (NEW) - report data object for the Caraga domain.
- lib/widgets/app_button.dart (UPDATED) - added isLoading so the submit button is disabled while a request is pending.
- lib/widgets/app_text_field.dart (UPDATED) - added maxLines, helperText and prefixIcon.
- lib/routes/app_routes.dart (UPDATED) - registered the /report route in the existing navigation shell.
- lib/screens/home_screen.dart (UPDATED) - added the 'Go to Report Form' button; existing navigation kept.
- lib/widgets/app_shell.dart (UPDATED) - added 'Report' to the top nav bar and the drawer.
- lib/constants/app_constants.dart (UPDATED) - report categories and Caraga locations.

The Activity 8 screens (Home, Login, Dashboard) were NOT replaced; the Report Form was added on top of the existing project and the same Git repository.

3. The Four Essential Async UI States

- LOADING - triggered by submitting a valid form while the simulated request is pending: the Submit button is disabled and shows a spinner ('Submit Report...') and the response area shows 'LOADING - Submitting your report'.
- SUCCESS - triggered when the simulated request resolves without error: a green confirmation with the saved report reference number and a 'Submit another report' action.
- EMPTY - triggered by opening the Report Form before any report record exists: 'EMPTY - No report records' with 'Submitted Reports (0)'. This is the no-data response, not a validation error.
- ERROR - triggered by turning on 'Simulate API failure (demo only)' and submitting: a red explanation of the failure plus a working Retry button that re-submits the form.

Duplicate submissions are prevented: the submit action returns early while the state is LOADING and the shared AppButton is disabled with a spinner during that time.

4. How to Run the Project

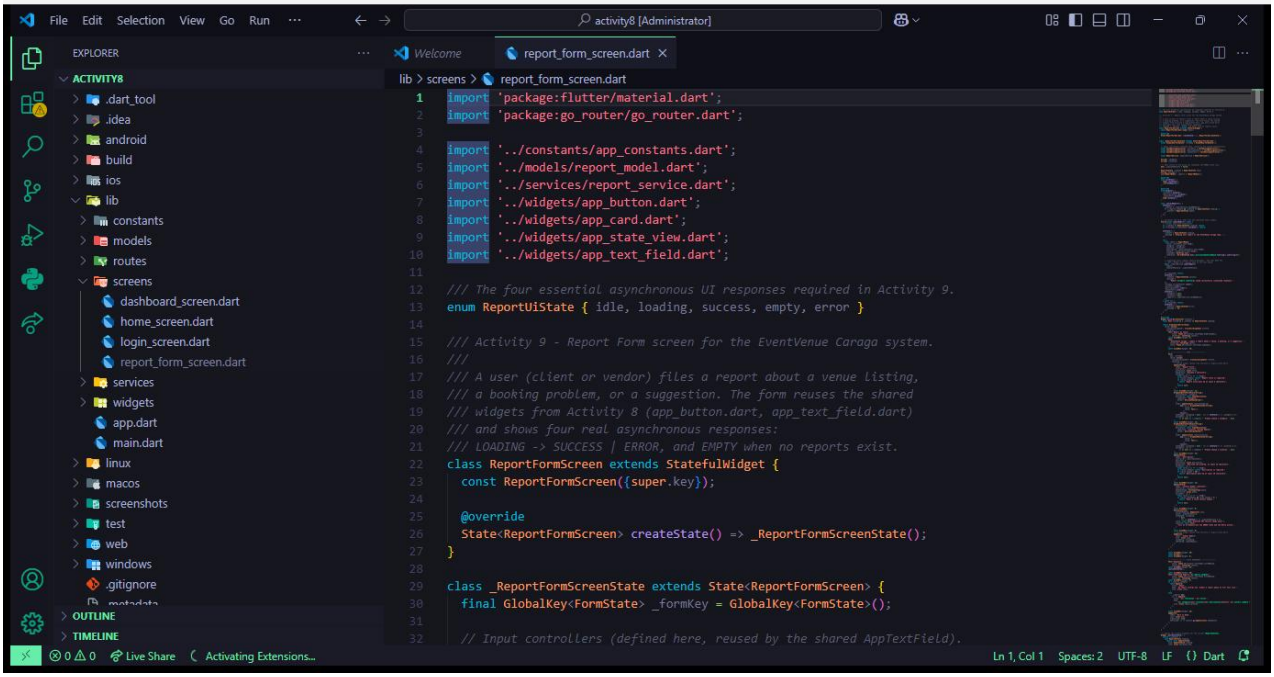
```
flutter pub get
flutter run -d chrome
flutter build web
```

Verification: flutter analyze -> No issues found; flutter test -> 6 tests passed (Home shell, Home -> Report Form navigation, EMPTY state, field-level validation, LOADING -> SUCCESS, ERROR -> Retry).

5. Mandatory Screenshot Evidence

Screenshot 1 - Updated screen / widget structure

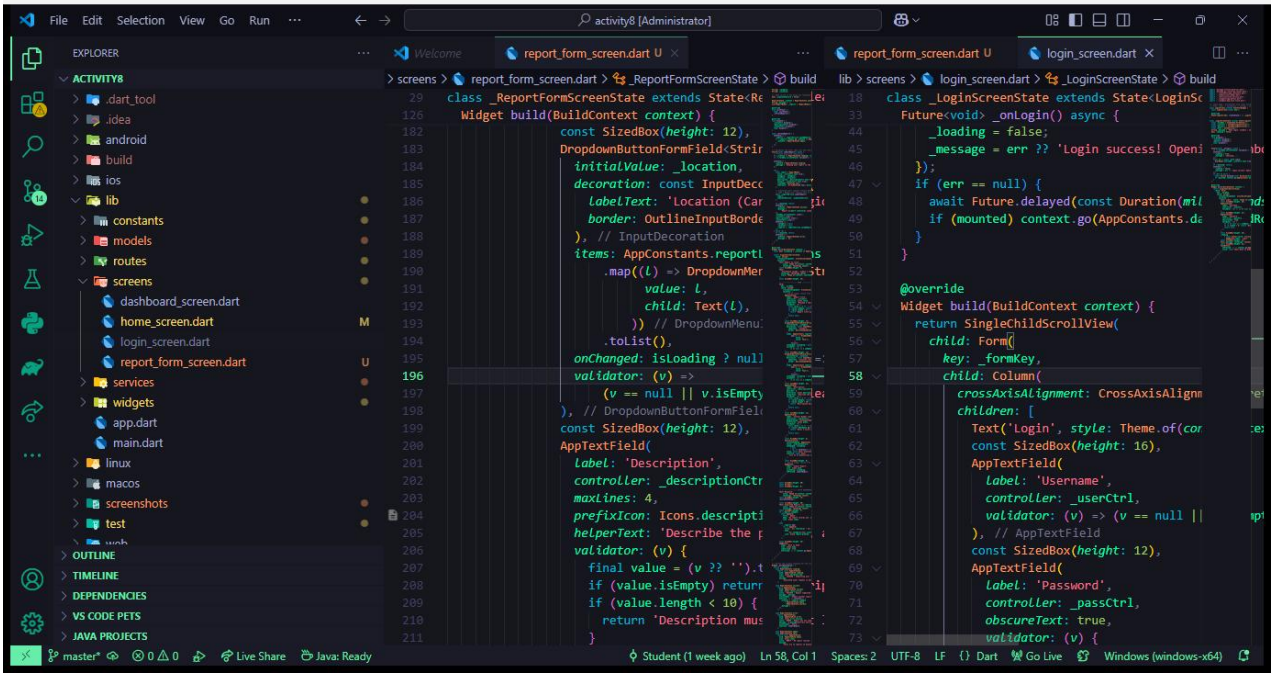
VS Code Explorer showing the Activity 8 folders (lib/screens, lib/widgets, lib/routes, lib/services, lib/models, lib/constants) together with the new lib/screens/report_form_screen.dart. The project name 'activity8' is visible in the Explorer and in the editor breadcrumb.



Screenshot 1 - VS Code Explorer with the Activity 8 structure and the new lib/screens/report_form_screen.dart

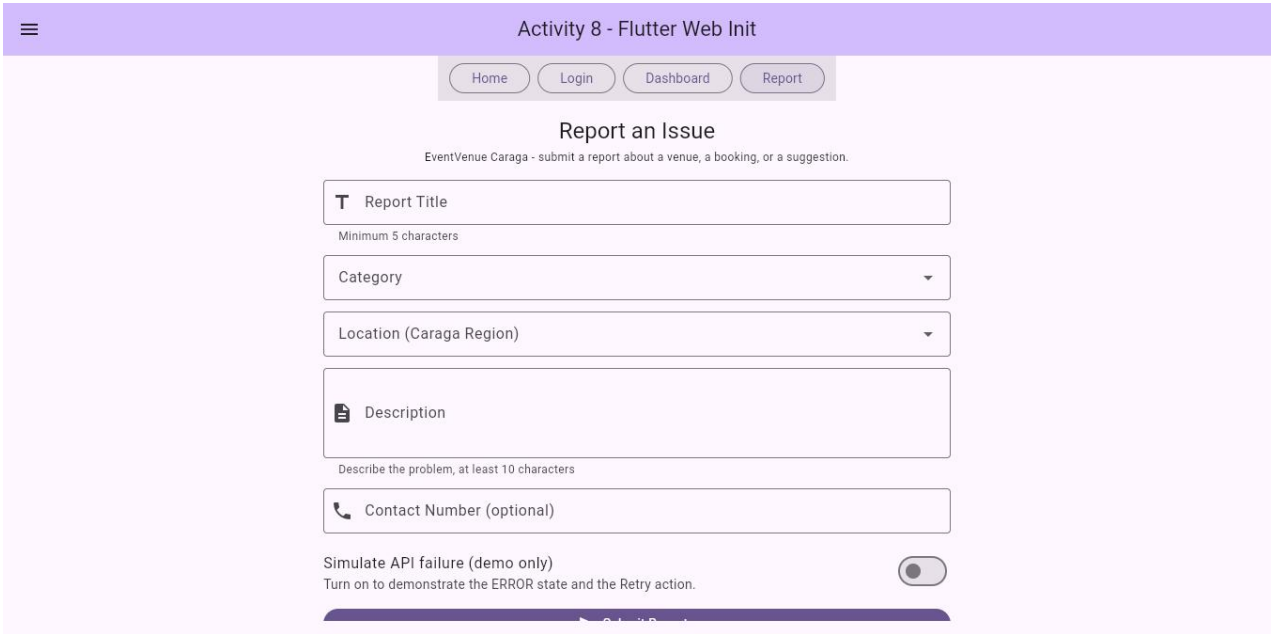
Screenshot 2 - Reusable widget used in two contexts

The shared AppTextField from lib/widgets/ is visible in two different screens of the same project: on the left, report_form_screen.dart (line 200, the Description field), and on the right, the Activity 8 login_screen.dart (line 63, the Username field). AppButton is likewise shared by home_screen.dart, login_screen.dart, dashboard_screen.dart and report_form_screen.dart.



Screenshot 2a - AppTextField from lib/widgets/ used in report_form_screen.dart (left) and in login_screen.dart (right)

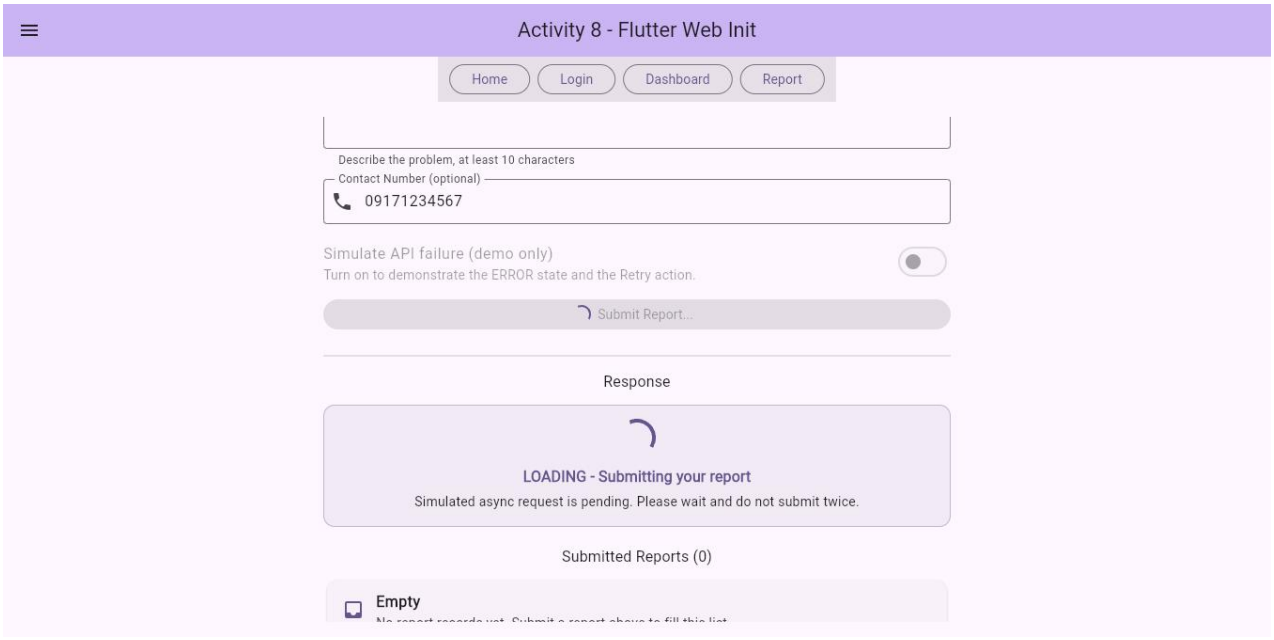
The same report_form_screen.dart running in Google Chrome, showing the real form fields built with the shared widgets (AppTextField and AppButton):



Screenshot 2b - report_form_screen.dart running in Google Chrome

Screenshot 3 - LOADING state

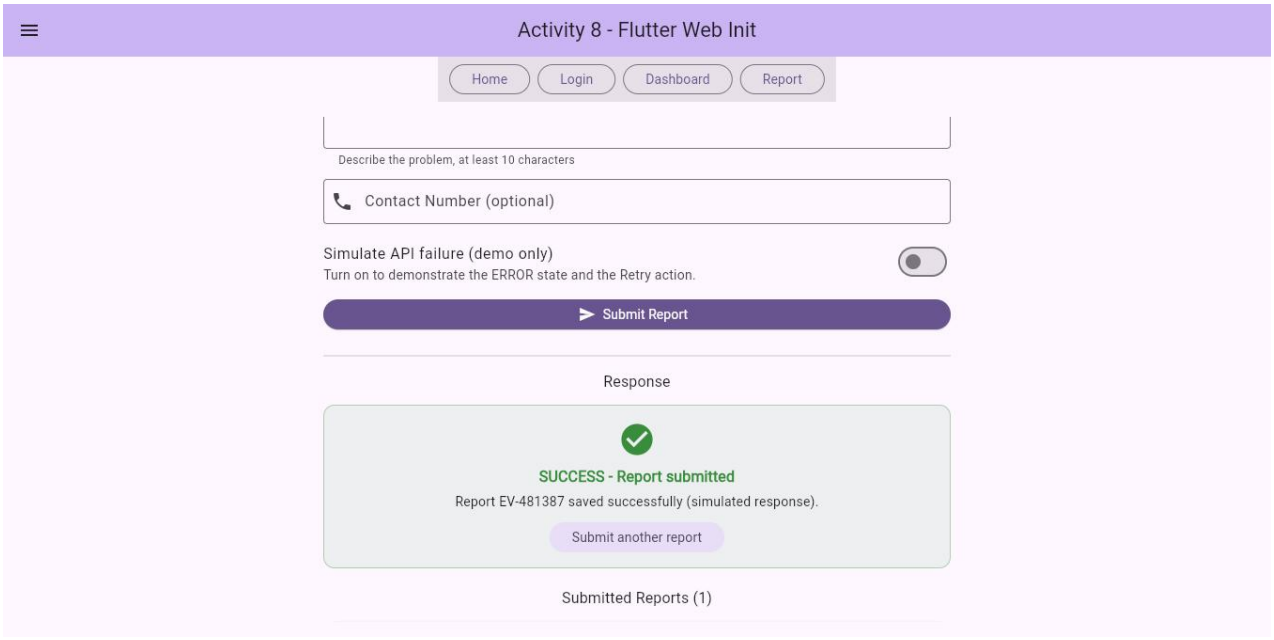
Trigger: the user filled in a valid report (title, category, location, description, contact) and pressed 'Submit Report'. The simulated asynchronous request is still pending, so the submit button is disabled with a spinner and the response area shows the LOADING message. This also proves that duplicate submission is prevented while the request is running.



Screenshot 3 - LOADING: button disabled with spinner, 'LOADING - Submitting your report'

Screenshot 4 - SUCCESS state

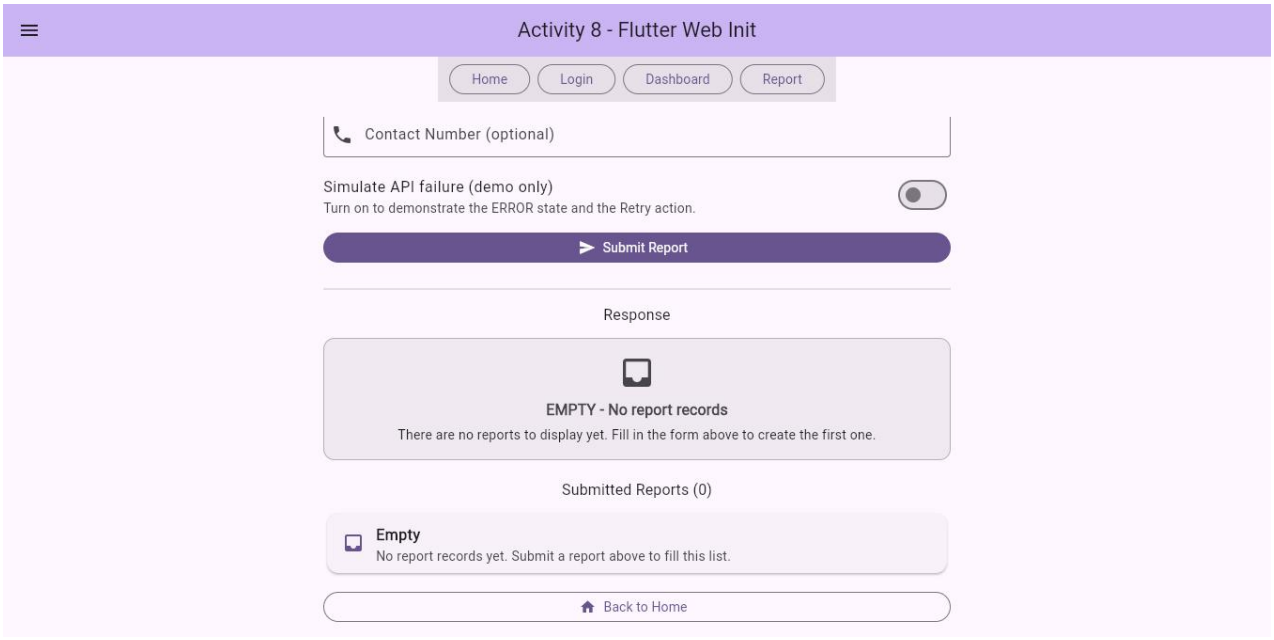
Trigger: the simulated request completed successfully (about 3 seconds after the press). The response area turns green with 'SUCCESS - Report submitted' and the saved reference number, and the submitted report now appears in the 'Submitted Reports' list below.



Screenshot 4 - SUCCESS: 'Report EV-481387 saved successfully (simulated response)' with a 'Submit another report' action

Screenshot 5 - EMPTY state

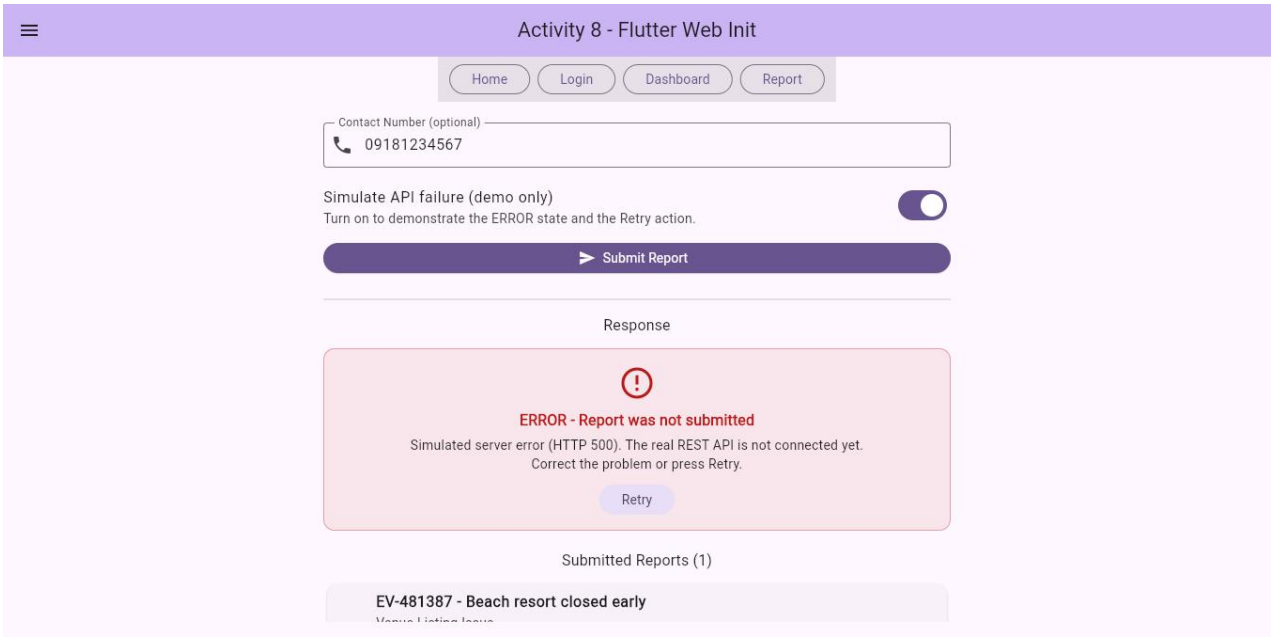
Trigger: the Report Form was opened before any report record existed. The response area shows 'EMPTY - No report records' and the list below shows 'Submitted Reports (0)'. This is the no-data response and is different from a validation error.



Screenshot 5 - EMPTY: 'EMPTY - No report records' and 'Submitted Reports (0)'

Screenshot 6 - ERROR state

Trigger: the user turned on the 'Simulate API failure (demo only)' switch and submitted a valid report. The simulated request failed, so the response area turned red with 'ERROR - Report was not submitted' and the reason. A visible Retry button is provided and re-runs the submission.



Screenshot 6 - ERROR: failure explanation with the visible Retry action

6. Functional Verification Checklist

Test	Observed outcome / evidence
Home -> Report Form navigation	Pass - the 'Go to Report Form' button on the Home screen and the 'Report' button in the navigation shell both open the Report Form (Screenshot 2b, Screenshot 1).
Invalid form produces field-level validation	Pass - empty title shows 'Report title is required', empty category 'Please choose a category', empty location 'Please choose a location', empty description 'Description is required'. Covered by flutter test ('Invalid input triggers field-level validation').
Valid submit -> Loading -> Success	Pass - Screenshot 3 (LOADING) then Screenshot 4 (SUCCESS). Covered by flutter test ('Valid submit goes LOADING then SUCCESS').
No available report records -> Empty	Pass - Screenshot 5 shows 'EMPTY - No report records' with 'Submitted Reports (0)'.
Simulated failure -> Error -> Retry	Pass - Screenshot 6 shows the ERROR explanation and the Retry button, which re-submits the form. Covered by flutter test ('Simulated failure shows ERROR with a Retry action').
Shared button or field widget used in two contexts	Pass - AppTextField is used in report_form_screen.dart (line 200) and in login_screen.dart (line 63) in the same project (Screenshot 2a). AppButton is used in 4 screens.

7. GitHub Checkpoints

The same repository and branch from Activity 8 was used. Activity 9 is the second checkpoint.

```
git status
git add .
git commit -m "Add report form and Flutter UI states"
git push
```

Checkpoint 1 (Activity 8): c7497c3 - Initial Flutter Web project - home/login/navigation shell

Checkpoint 2 (Activity 9): 1a14bf0 - Update group member to andymarkcrasco

Checkpoint 3 (Activity 9): 1ef6ac3 - Add report form and Flutter UI states

GitHub repository link: https://github.com/andymarkcrasco-blip/activity8

8. Notes and Limitations

- The four UI states are SIMULATED. The asynchronous delay is produced by Future.delayed in lib/services/report_service.dart because the REST API (instructor-approved backend, e.g. FastAPI) and the PostgreSQL database (IT 108) are not connected yet.
- The ERROR state is reproduced on purpose with the 'Simulate API failure (demo only)' switch; the switch is clearly labeled so the response is not mistaken for a real crash.
- When the real API is connected, only report_service.dart has to change: replace the Future.delayed block with an http.post call, map the JSON into ReportModel, and the four states keep working because they are driven by the same ReportUiState enum.

- No report record is actually saved in a database in this activity - the saved list is kept in memory for the running session.

9. Post-Laboratory Reflection

1. How does report_form_screen.dart extend your Activity 8 project without replacing its existing Home and Login screens?

report_form_screen.dart was added as a new file under lib/screens/ and registered as a new '/report' route in the existing go_router configuration (lib/routes/app_routes.dart). The Activity 8 Home, Login and Dashboard screens were only given an extra navigation button; their original code and behavior were not replaced. The folder structure from Activity 8 (screens, widgets, routes, services, models, constants) is preserved, so the new screen simply reuses the same shared components instead of duplicating interface code.

2. Which shared widget did your group reuse, and what benefit does that provide?

The group reused AppTextField from lib/widgets/app_text_field.dart in two different screens - the Description field of report_form_screen.dart (line 200) and the Username field of login_screen.dart (line 63) - and AppButton in four screens (home, login, dashboard and report form). The benefit is the 'build once, reuse many times' principle: the label style, borders, validation hooks and button behavior stay consistent across the whole application, any change to the component automatically updates every screen that uses it, there is much less duplicated code, and each team member can work on a different layer without conflicting edits.

3. How does your report form prevent invalid input and duplicate submissions?

Invalid input is prevented by a GlobalKey<FormState> and per-field validators: the report title needs at least 5 characters, the description at least 10 characters, the category and the location are required dropdowns, and an optional contact number must be at least 7 digits. Duplicate submission is prevented in two ways: the submitReport() method returns immediately while the state is LOADING, and the shared AppButton is given isLoading: true so it is rendered disabled with a spinner while the simulated request is still pending.

4. How did you reproduce the Loading, Success, Empty, and Error responses? Which were simulated?

LOADING was reproduced by submitting a valid form and capturing the screen while the asynchronous request was still pending (the button was disabled with a spinner). SUCCESS was reproduced by letting the request finish without an error, which returned the green confirmation with the report reference number. EMPTY was reproduced by opening the Report Form before any report record existed, showing the 'EMPTY - No report records' response and 'Submitted Reports (0)'. ERROR was reproduced by turning on the 'Simulate API failure (demo only)' switch and submitting, which produced the red failure explanation and the Retry button. ALL FOUR responses are currently SIMULATED using Future.delayed, because the REST API and the PostgreSQL database are not connected yet.

5. What must be changed when the real REST API and PostgreSQL database are connected later?

Only lib/services/report_service.dart has to change: the Future.delayed block is replaced with an http.post call to the instructor-approved REST API (e.g. FastAPI), the JSON response is mapped into the ReportModel, and the screen loads the report list from the API so that the EMPTY state is produced by a real query instead of an empty in-memory list. The ReportUiState enum, the AppStateView widget and all four state builders in report_form_screen.dart stay exactly the same, so the LOADING, SUCCESS, EMPTY and ERROR responses keep working with real data; the 'Simulate API failure' switch can then be removed because genuine server errors will be handled by the same catch block.